

NeuFlow v3: Queryable Optical Flow with Calibrated Uncertainty

Shriman Raghav Srinivasan

MS Robotics, Northeastern University · Field Robotics Lab

August 2026

Abstract

NeuFlow v3 replaces the fixed convex upsampler of NeuFlow v2 with an implicit decoder that evaluates optical flow at arbitrary continuous coordinates. The network answers queries rather than emitting a fixed-resolution map: cost scales with the number of points requested, $O(N)$, instead of image area, $O(H \times W)$.

On VKITTI2 the decoder matches v2’s accuracy using 13% fewer parameters, and adds three capabilities the fixed-resolution architecture cannot express: flow at sub-pixel coordinates, repeat queries against a cached frame at $7.7\times$ lower cost, and a calibrated per-query confidence estimate. The intended application is registration and mapping, where a consumer needs a few hundred correspondences at points of its own choosing rather than half a million it discards.

1 Motivation

Optical flow networks compute displacement for every pixel at a single fixed resolution. Many consumers of flow do not need that product:

- **Image registration** requires a few hundred correspondences at well-textured locations it selects itself.
- **Sparse tracking** needs flow only at feature points.
- **Survey mosaicking** needs matches inside overlap regions, frequently at sub-pixel positions.

On constrained hardware, computing 479,232 values in order to consume 800 of them determines whether a system runs in real time. The fixed-resolution design also forecloses any query above the input sampling rate, which is precisely what high-resolution registration requires.

NeuFlow v3 asks whether the final stage of a real-time flow network can be made queryable without surrendering accuracy or throughput.

2 Method

2.1 Inherited pipeline

NeuFlow v2 (Zhang, Gupta, Jiang & Singh, arXiv:2408.10161) proceeds as: a shallow CNN backbone producing features at $1/8$ and $1/16$ resolution; cross-attention and global matching at $1/16$ to establish an initial flow; recurrent refinement (one iteration at $1/16$, eight at $1/8$); and a learned convex upsampler mapping $1/8$ resolution flow to full resolution.

v3 retains everything up to and including the $1/8$ -resolution coarse flow, with all learned weights frozen. Only the upsampler is replaced. In the runs reported here the batch-normalisation *running statistics* of the frozen stack adapted to the training data rather than remaining at v2’s values (see Section 6); the learned weights are unchanged.

2.2 Implicit decoder

The decoder operates in two phases.

Phase 1 — once per frame pair (16.6 ms). The frozen pipeline produces the coarse flow and feature maps, which are cached.

Phase 2 — once per query batch (2.55 ms for up to 2,048 points). For a query coordinate (x, y) , the decoder samples 3×3 windows from four sources — 1/8 context, 1/8 features, 1/16 features, and flow-warped frame-1 features — fuses them through a gated hierarchical MLP, and predicts weights over ten candidates: the nine coarse-flow values in the local neighbourhood plus a bilinear sample.

The output is a **convex combination** of those candidates. This bounds the prediction inside the locally supported motion range, so the decoder cannot hallucinate displacement unsupported by its inputs. The head is initialised so that the softmax concentrates on the bilinear candidate, making the untrained decoder exactly equivalent to bilinear upsampling (verified: 0.011 px maximum deviation). Training therefore begins from a known-good operating point and can only depart from it by learning.

2.3 Uncertainty head

An optional eleventh output predicts a per-query error scale b , trained under a Laplace likelihood alongside the flow loss. It costs no measurable inference time and yields a confidence value for every returned correspondence.

2.4 Interface

```
state = model.infer_coarse_state(img0, img1)           # once per pair
flow  = model.decode_queries(state, query_coords=q)    # q: [B, N, 2] pixels
flow  = model.decode_queries(state, target_h=H, target_w=W)
flow, b = model.decode_queries(state, query_coords=q,
                               return_uncertainty=True)
```

Coordinates are continuous: (312.7, 188.2) is as valid an argument as (312, 188). Sparse queries reproduce the dense field exactly at the same coordinates (0.00057 px maximum difference).

3 Experimental setup

Evaluation. VKITTI2 Scene18 and Scene20, 1,174 frame pairs, 460,573,660 valid pixels, per-pixel metrics. These scenes are excluded from every training set, enforced by a guard in the data loader and verified at both pair and frame level before each run.

Controls. All training runs are generated from a single template so that any two differ in exactly one variable. Every run uses seed 1234, batch size 16, learning rate 2×10^{-4} under a OneCycle schedule, loss weighting $\gamma = 0.8$, 4,096 supervision queries per image, a refinement schedule of $1 \times s16 + 8 \times s8$ matching the evaluation configuration, and 100,000 steps. The backbone’s learned weights are frozen throughout. Batch-normalisation running statistics adapted during these runs; Section 6 quantifies the effect.

Pre-flight verification. A nine-check suite runs on both CPU and GPU before any training job: batch-normalisation frozen, only decoder parameters trainable, zero-initialised decoder equals bilinear upsampling, sparse queries equal dense output, uncertainty available in both decode paths, stride-2 decoding fidelity, and seed reproducibility.

Hardware. NVIDIA V100, fp16, 384×1248 input unless stated.

4 Results

4.1 Accuracy

Model	Training data	EPE (px)	1px (%)	3px (%)	Params
NeuFlow v2	FlyingThings	2.324	77.63	89.80	9.03 M
NeuFlow v3	FlyingChairs	2.286	71.30	87.57	7.83 M
NeuFlow v3	+ VKITTI2	2.138	76.38	89.46	7.83 M
NeuFlow v3	+ MPI-Sintel	2.147	76.81	89.56	7.83 M
NeuFlow v3	+ uncertainty head	2.104	76.88	89.61	7.83 M

v3 matches v2’s accuracy with 13% fewer parameters. Trained on FlyingChairs alone — containing no driving imagery, mirroring v2’s own training, which likewise contained none — v3 reaches 2.286 px against v2’s 2.324 px on unseen driving scenes. This is the like-for-like comparison and the two models are equivalent on it.

Trained with in-domain data, v3 reaches 2.104 px. That configuration sees VKITTI2 scenes from the same simulator as the test set, which v2’s training did not, so it measures what the architecture achieves given representative data rather than a like-for-like advantage.

The three mixed-data configurations span 2.10–2.16 px and are not separable from one another: evaluating two checkpoints of each run shows between-checkpoint variation of up to 0.038 px, comparable to the differences between them. Differences below approximately 0.05 px are not resolved by this experiment. The effect that is robust across checkpoints is the addition of in-domain data, worth roughly 0.15 px and five points of 1-pixel accuracy.

4.2 Compute

Mode	Latency	Note
NeuFlow v2, full frame	19.6 ms	its only mode
v3, sparse, first query on a new pair	19.16 ms	equivalent to v2
v3, repeat query on a cached pair	2.55 ms	7.7× cheaper
v3, dense output, stride 2	22.0 ms	not the intended mode

The sparse path divides into a 16.61 ms coarse pass, inherited unchanged from v2, and a 2.55 ms decode. Because global matching requires whole-image context, the coarse pass is irreducible; it accounts for 87% of the first-query cost, which is why a first query costs what a v2 frame costs.

The decisive property is state reuse. v2 retains nothing between calls, so a second question about the same frame requires recomputing everything. v3 answers it from cached state in 2.55 ms. For any consumer that queries a frame more than once — iterative registration, RANSAC refinement, interactive selection, multi-hypothesis tracking — this is a structural difference rather than a margin.

Decode latency is 2.553 ms at $N = 800$ and 2.554 ms at $N = 2,048$: the decode is launch-overhead bound rather than compute bound over this range, so 2,048 points cost the same as 800.

4.3 Calibrated uncertainty

Predicted scale b	Actual mean error
0.01 – 0.11	0.313 px
0.11 – 0.20	0.504 px
0.20 – 0.41	0.807 px
0.41 – 1.22	1.652 px
> 1.22	6.723 px

Measured over 2,348,000 queries. Predicted uncertainty rises monotonically with actual error across all five bins, spanning a factor of 21, with Pearson correlation 0.345. The signal is directly usable: weighting correspondences in RANSAC, rejecting unreliable matches, or allocating additional queries to uncertain regions. NeuFlow v2 emits flow alone and offers no comparable quantity.

4.4 Interactive tool

A PyQt application loads a video, steps frame by frame, and computes flow for a user-drawn region. Two modes are provided: exact decoding restricted to the selected region against a full-frame coarse pass, and a cropped mode in which the entire pipeline runs on the selection so that cost scales with the requested area. Both report their latency breakdown live.

5 Discussion

The contribution is an operating point rather than a leaderboard position. v3 delivers v2’s accuracy from a smaller model while adding three properties the fixed-resolution design cannot express:

1. **Arbitrary-coordinate access.** Flow at any continuous position, with sparse output identical to dense output at matching coordinates.
2. **Amortised repeat access.** 2.55 ms per additional query batch against a cached frame, versus a full 19.6 ms recomputation.
3. **Per-query confidence.** A calibrated error estimate accompanying every correspondence.

For an application that consumes a complete dense flow field exactly once per frame, NeuFlow v2 remains the appropriate choice: it produces that product 12% faster and with higher sub-pixel precision. v3 is the appropriate choice when the consumer selects its own query points, revisits a frame, needs positions between pixels, or requires a confidence value.

6 Limitations

Sub-pixel precision. v3 trails v2 on 1-pixel accuracy by 0.8 to 6.3 points depending on configuration, while matching or exceeding it on mean error — the decoder produces fewer large errors but is less exact on the majority of pixels. The cause is identified: the decoder’s finest input is at 1/8 resolution, so the evidence available to it varies little within an 8×8 cell, whereas v2’s upsampler reads the full-resolution frame directly. A Fourier positional encoding of the sub-cell offset produced no change, establishing that the limitation is missing high-resolution *features* rather than missing positional information.

Edge validation. All latency figures are from V100 and RTX 4060 hardware. No measurement on an embedded target has been taken.

Evaluation breadth. Accuracy is measured on VKITTI2, a synthetic driving benchmark. Generalisation to field and survey imagery is untested.

Statistical resolution. One seed per configuration, with checkpoint-to-checkpoint variation of up to 0.038 px. Differences below roughly 0.05 px are not resolved.

Adapted normalisation statistics. The frozen stack’s learned weights are unchanged, but its batch-normalisation running statistics accumulated approximately 24,800 updates during training rather than staying at v2’s values. Two consequences follow. First, the comparison in Section 4.1 is between v3 with statistics adapted to its own training set and v2 with statistics from FlyingThings, rather than a decoder-only comparison; for the mixed-data configurations that adaptation points toward the evaluation domain. Second, out-of-domain robustness suffers: on 1920×1080 Spring imagery, far from the 320×496 training crops, the adapted statistics degrade the coarse flow badly while v2’s untouched statistics do not. Runs with strictly frozen statistics are required to separate the decoder’s contribution from this effect.

Spring evaluation. Evaluation against Spring’s $2\times$ resolution ground truth, which would test querying above the input sampling rate, is pending the re-run described above.

7 Future work

1. **Full-resolution stem.** Supplying the decoder with a cheap full-resolution feature map (estimated 1–2 ms) directly addresses the identified precision limitation and would make sub-pixel querying substantive rather than interpolative.
2. **Decode-path optimisation.** The decode is launch-overhead bound; CUDA graph capture, kernel fusion of the three coordinate-sharing samplers, and `torch.compile` (measured at 9% on the coarse pass) together project a first-query latency below v2’s.
3. **Spring high-resolution evaluation,** testing output above the input sampling rate.
4. **Embedded measurement** on Jetson-class hardware.
5. **Registration on field imagery,** evaluating the intended application directly.

Code, training configurations and a complete chronological development record are maintained at github.com/shrirag10/Neuflowv3 (branch v3-dev). Every figure in this report is reproducible from `scripts/eval.all_runs.py`, `scripts/benchmark.sparse.py` and `scripts/eval.calibration.py`.